S01 Health and Damage

CRE132 worksheet

What you'll make

A **Health** component that anything in a game can have: the player, enemies, breakable crates. Then you'll add the things that use it: a fire that hurts, a potion that heals, an attack, and a tint that turns characters red as they get hurt.

By the end you'll know how one script calls another (`GetComponent` and public methods), why some things are **public** and others **private**, and how triggers find out what touched them.

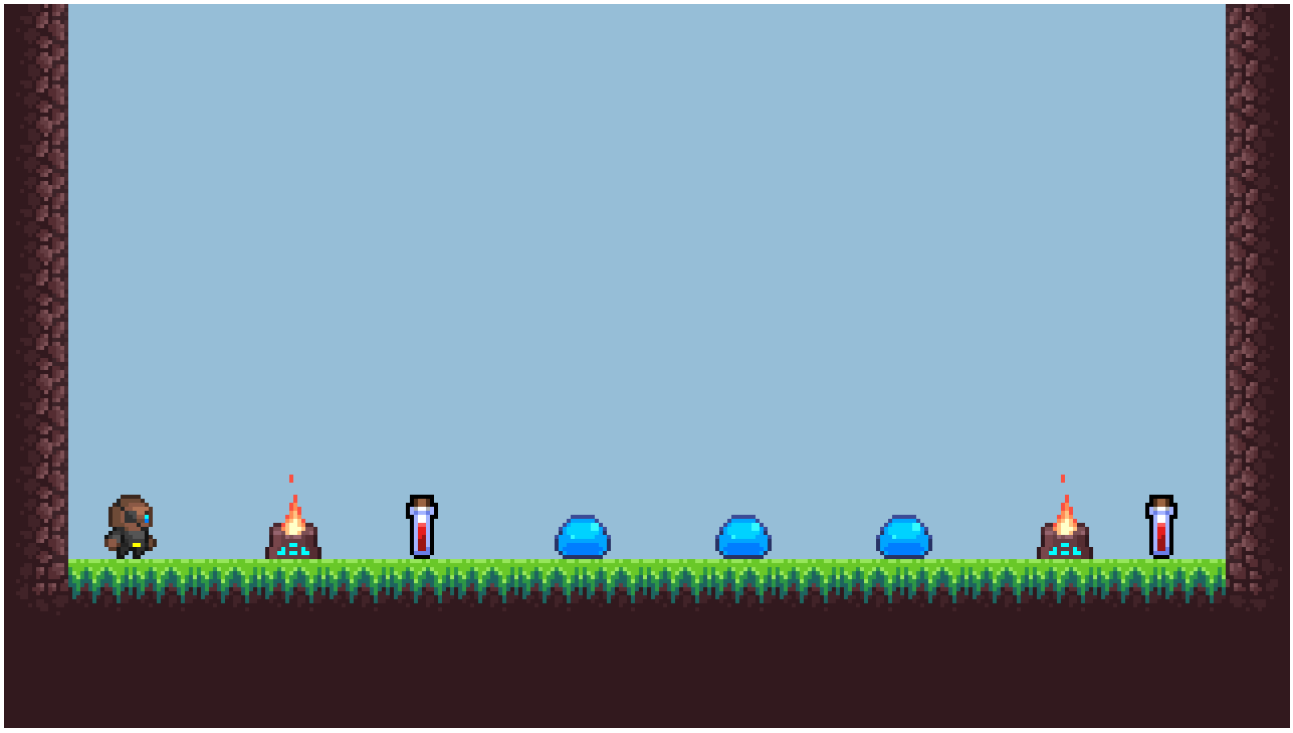


Figure 1: The finished scene: the player, three training dummies, two fires and two potions.

Before you start

1. Unzip `CRE132_S01_HealthAndDamage_Starter.zip`. In Unity Hub, choose **Add > Add project from disk** and pick the `CRE132_S01_HealthAndDamage_Starter` folder. Open it with Unity **6000.3.11f1**.
2. Open the scene `Assets/_game/Scenes/Shared/S01_HealthAndDamage`.
3. Press **Play**. You can walk (A/D or arrows), run (hold Shift) and jump (Space), but nothing can be hurt yet. Press Play again to stop.

The scene has the **Player**, three **Training Dummies**, two fires called **Fire (Damage Zone)** and two **Health Potions**. You'll give them their scripts.

> **Making a script:** in the Project window, go to the folder, right-click and choose **Create > Scripting > MonoBehaviour Script**. Type the name straight away and press Enter. The name must match the class name in the code exactly, capital letters included.

1. The Health script

Make a folder `Assets/_game/Scripts/Health` and create a script in it called `Health`. Double-click it to open it, and replace everything with:

```
// Health that anything can have: the player, enemies, breakable crates.
// Other scripts call TakeDamage and Heal; this script decides what happens at zero.
using UnityEngine;

public class Health : MonoBehaviour
{
    [SerializeField] private int maxHealth = 3;

    private int currentHealth;

    // Read-only from outside: other scripts can look at health but must use
    // TakeDamage or Heal to change it.
    public int CurrentHealth
    {
        get { return currentHealth; }
    }

    public int MaxHealth
    {
        get { return maxHealth; }
    }

    private void Awake()
    {
        currentHealth = maxHealth;
    }

    public void TakeDamage(int amount)
    {
        // Already dead: ignore any more hits that arrive in the same frame.
        if (currentHealth <= 0)
        {
            return;
        }

        currentHealth = currentHealth - amount;
        if (currentHealth < 0)
        {
            currentHealth = 0;
        }
        Debug.Log(gameObject.name + " took " + amount + " damage, health " +
            currentHealth + "/" + maxHealth);

        if (currentHealth == 0)
        {
            Die();
        }
    }

    public void Heal(int amount)
    {
        currentHealth = currentHealth + amount;
        if (currentHealth > maxHealth)
        {
            currentHealth = maxHealth;
```

```
        }
        Debug.Log(gameObject.name + " healed " + amount + ", health " +
            currentHealth + "/" + maxHealth);
    }

    private void Die()
    {
        Debug.Log(gameObject.name + " died");
        Destroy(gameObject);
    }
}
```

What's going on:

- **`[SerializeField] private`** shows `maxHealth` in the Inspector, so each object can have its own value, while other scripts still can't change it.
- **`currentHealth` is set in `Awake`**, which runs once when the object appears, so every object starts at full health.
- **`TakeDamage` and `Heal` are public**: other scripts call them. The number itself stays private. `CurrentHealth` and `MaxHealth` are **properties** with only a `get`, so other scripts can read them but not change them.
- **The `if` checks** stop health going below 0 or above the maximum.
- **`Die` is private**: only Health decides when something dies. For now it just destroys the object; later samples will play animations or respawn the player instead.

Save the script (Ctrl+S) and go back to Unity. If the Console shows red errors, compare your code with the listing above, line by line.

Add it: select the **Player** and click **Add Component > Health**. Set **Max Health** to **5**. Then select all three **Training Dummies** at once (click the first, Shift-click the last) and add **Health** to all of them; leave Max Health at 3.

2. A fire that hurts: DamageZone

Create a script `DamageZone` in the same `Health` folder:

```
// Hurts anything with a Health component while it stays inside this trigger: fire,
// spikes, lava.
using UnityEngine;

public class DamageZone : MonoBehaviour
{
    [SerializeField] private int damage = 1;
    [SerializeField] private float timeBetweenHits = 1f;

    // The time when this zone may hurt again. Without it, OnTriggerStay2D would hit
    // every physics step.
    private float nextHitTime;

    private void OnTriggerStay2D(Collider2D other)
    {
        if (Time.time < nextHitTime)
        {
            return;
        }

        Health health = other.GetComponent<Health>();
        if (health != null)
```

```
        {
            health.TakeDamage(damage);
            nextHitTime = Time.time + timeBetweenHits;
        }
    }
}
```

- **`OnTriggerStay2D`** runs every physics step while something is inside the trigger. The timer (`nextHitTime`) makes it hurt once a second instead.
- **`other.GetComponent<Health>()`** asks the thing in the fire for its Health. The ground has none, so we get `null`, and the **`!= null` check** skips it. The fire doesn't need to know *what* walked in, only whether it has Health.

Add it to both **Fire (Damage Zone)** objects (select both, Add Component). Their Box Collider 2D already has **Is Trigger** ticked.

Test it: press Play, stand in a fire and watch the **Console**: one hit a second. Five hits and the player is destroyed.

> **Why the player keeps getting hurt while standing still:** a Rigidbody2D that stops moving can go to *sleep* to save work, and sleeping bodies get no `OnTriggerStay2D`. The Player's Rigidbody 2D has **Sleeping Mode** set to **Never Sleep** to stop that.

3. A potion that heals: HealthPickup

Create `HealthPickup` in the `Health` folder:

```
// A pickup that heals whatever touches it, then disappears.
using UnityEngine;

public class HealthPickup : MonoBehaviour
{
    [SerializeField] private int healAmount = 2;

    private void OnTriggerEnter2D(Collider2D other)
    {
        Health health = other.GetComponent<Health>();
        if (health != null)
        {
            health.Heal(healAmount);
            Destroy(gameObject);
        }
    }
}
```

The same pattern as the fire, but with **`OnTriggerEnter2D`**, which runs once when something arrives. `Destroy(gameObject)` removes the potion itself.

Add it to both **Health Potions**, then test: get hurt in the fire, then walk into a potion. The Console shows the heal, and health never goes above 5.

4. Hitting things: SimpleAttack

To test Health you need something that hits. A proper melee attack, with animation and a cooldown, comes in T04, so for now you get a ready-made test script.

First, an Attack action. Double-click `CRE132Controls` (in `Assets/_game/Input`). Click **+** next to **Actions** and name the new action `Attack`; leave its **Action Type** as **Button**. Select the binding under it (**<No**

Binding>), click the **Path** drop-down, press **Listen** and press **J**. Then add two more with **+ > Add Binding**: **Left Button [Mouse]** and **Button East [Gamepad]** (B on Xbox, Circle on PlayStation). Click **Save Asset** at the top of the window (or tick **Auto-Save**) and close the editor.

Then the script. Make a folder `Assets/_game/Scripts/S01_HealthAndDamage`. Your Starter project has a `Provided` folder next to `Assets`, with `SimpleAttack.cs` in it. Drag the file from your file manager into the new folder in Unity's Project window. (It had to wait outside `Assets` because it uses your Health script, which didn't exist until now.) Double-click it to read along:

- **`InputActionReference attackAction`** is the same kind of slot as C11's Move and Jump. `WasPressedThisFrame()` is true only on the frame the button goes down, so holding J hits once.
- **`Physics2D.OverlapCircleAll`** returns every collider inside a circle just in front of the player.
- **`continue`** skips the rest of the loop for that collider: the circle also touches the player's own collider, and we don't want to hit ourselves.
- **`GetComponent<Health>()`** finds your Health script on whatever was hit, and `TakeDamage` does the rest. Anything without Health (walls, the floor) is ignored.
- **`GetHitPoint`** puts the circle in front of the player: to the left when the Sprite Renderer's **Flip X** is ticked.
- **The gizmo** draws the red circle in the Scene view while the Player is selected, so you can see where the attack lands.

Add it to the **Player**. In the **Attack Action** slot, click the small circle and pick **Player/Attack** (from `CRE132Controls`).

Test it: walk up to a dummy and press **J**. Three hits and it's gone.

5. Showing damage: HealthTint

Create `HealthTint` in the `S01_HealthAndDamage` folder:

```
// Turns the sprite redder as health drops, so damage shows without any UI (S03 adds a
// health bar).
using UnityEngine;

public class HealthTint : MonoBehaviour
{
    [SerializeField] private Color hurtColour = new Color(1f, 0.25f, 0.25f);

    private Health health;
    private SpriteRenderer spriteRenderer;
    private Color healthyColour;

    private void Awake()
    {
        health = GetComponent<Health>();
        spriteRenderer = GetComponent<SpriteRenderer>();
        healthyColour = spriteRenderer.color;
    }

    private void Update()
    {
        // 1 at full health, 0 at none. The cast to float stops whole-number
        // division (2 / 3 would be 0).
        float healthFraction = (float)health.CurrentHealth / health.MaxHealth;
        spriteRenderer.color = Color.Lerp(hurtColour, healthyColour, healthFraction);
    }
}
```

- **`(float)`** turns the health into a decimal number before dividing. Without it, C# divides whole numbers and throws away the remainder: `2 / 3` is `0`.
- **`Color.Lerp`** blends between two colours: 0 gives the hurt colour, 1 the normal colour, 0.5 halfway.

Add it to the **Player** and the three **Training Dummies**, then play: everything gets redder as it's hurt.

Check your work

- ☐ Standing in a fire costs 1 health a second, even when standing still.
- ☐ A potion heals 2, never above the maximum, and disappears.
- ☐ `CRE132Controls` has an **Attack** action bound to J, the left mouse button and Button East.
- ☐ J hits the dummy in front of you, in whichever direction you face. Three hits destroy it.
- ☐ Hurt things turn redder. At zero health the player is destroyed.
- ☐ Compare with `CRE132_S01_HealthAndDamage_Finished` if something behaves differently.

Key points

- **Scripts talk through public methods.** `GetComponent<Health>()` finds another script on the same object, and `TakeDamage` is public so it can be called. Always check for `null` first.
- **The attacker doesn't need to know what it hit**, only that it has a Health component. One Health script works for the player, the dummies, and anything else you make later.
- **Triggers** only fire when at least one of the two objects has a **Rigidbody2D**. The dummies have none, so the fire never hurts them.

If something goes wrong

- **"The type or namespace name 'Health' could not be found":** the file and the class have different names, or the Health script has an error of its own. Fix the first red error in the Console first.
- **The script won't go on an object ("no MonoBehaviour scripts in the file"):** the class name doesn't match the file name.
- **NullReferenceException in SimpleAttack:** the Attack Action slot is empty.
- **NullReferenceException in HealthTint:** the object has HealthTint but no Health.
- **The fire does nothing:** check that the fire's Box Collider 2D has **Is Trigger** ticked.

Try it

1. Make one fire stronger: **Damage** 2 and **Time Between Hits** 0.5.
2. Give a dummy a **Rigidbody 2D** and push it into the fire. Does it get hurt now? Why?
3. Set the Player's **Sleeping Mode** back to **Start Awake**, stand still in the fire and watch the damage stop.
4. Make a **Health Potion** that hurts instead of heals, without writing any new code. (Hint: what happens if `healAmount` is negative? Is that a good idea?)